Figure 4.11 In the double-submit cookie pattern, the server avoids storing a second token by setting it as a second cookie on the client. When the legitimate client makes a request, it reads the CSRF cookie value (which cannot be marked HttpOnly) and sends it as an additional header. The server checks that the CSRF cookie matches the header. A malicious client on another origin is not able to read the CSRF cookie and so cannot make requests. But if the attacker compromises a sub-domain, they can overwrite the CSRF cookie with a known value.

The solution to these problems is to make the second token be *cryptographically bound* to the real session cookie.

> **DEFINITION** We say that some object is ***cryptographically bound*** to another object if it is computationally infeasible to change one of the objects without a detectable change in the other.

Rather than generating a second random cookie, you will run the original session cookie through a *cryptographically secure hash function* to generate the second token. This ensures that any attempt to change either the anti-CSRF token or the session cookie will be detected because the hash of the session cookie will no longer match the token. As the attacker cannot read the session cookie, they are unable to compute the correct hash value. Figure 4.12 shows the updated double-submit cookie pattern. Unlike the password hashes used in chapter 3, the input to the hash function is an unguessable string with high entropy. You therefore do not need to worry about slowing the hash